

```

#include<iostream>
using namespace std;

struct Node
{
    float marks = 0;
    Node* next = nullptr;
};

class SLL
{
    long list_size = 0;
    Node* starting_node = nullptr;
public:
    SLL(void) {}
    ~SLL(void);
    void print(void);
    void push_front(float marks_to_add);
    void push_back(float marks_to_add);
    void delete_kth(long index);
    float at(long index);
    void swap_all(void);
};

void SLL::swap_all(void) {
    if (list_size < 2) {
        return;
    }

    Node* current_node = starting_node;
    Node* prev_node = nullptr;

    for (; current_node != nullptr && current_node->next != nullptr; current_node
= current_node->next) {
        Node* next_node = current_node->next;
        current_node->next = next_node->next;
        next_node->next = current_node;

        if (prev_node == nullptr) {
            starting_node = next_node;
        }
        else {
            prev_node->next = next_node;
        }

        prev_node = current_node;
    }
}

float SLL::at(long index)
{
    Node* current_node = starting_node;
    for (long i = 1; i < index; i++)
    {
        current_node = current_node->next;
    }
    return current_node->marks;
}

```

```

void SLL::delete_kth(long index)
{
    if (index == 1)
    {
        Node* node_to_delete = starting_node;
        starting_node = starting_node->next;
        delete node_to_delete;
    }
    else
    {
        Node* current_node = starting_node;
        for (long i = 1; i < index - 1; i++)
        {
            current_node = current_node->next;
        }
        Node* node_to_delete = current_node->next;
        current_node->next = node_to_delete->next;
        delete node_to_delete;
    }
    list_size--;
}

void SLL::push_back(float marks_to_add)
{
    cout << " Adding to back: " << marks_to_add << endl;
    Node* new_node = new Node;
    new_node->marks = marks_to_add;
    new_node->next = nullptr;
    if (list_size == 0)
    {
        starting_node = new_node;
    }
    else
    {
        Node* current_node = starting_node;
        while (current_node->next != nullptr)
        {
            current_node = current_node->next;
        }
        current_node->next = new_node;
    }
    list_size++;
}

SLL::~SLL(void)
{
    Node* next_node = nullptr;
    for (long i = 1; i <= list_size; i++)
    {
        next_node = starting_node->next;
        cout << " Deleting: " << starting_node->marks << endl;
        delete starting_node;
        starting_node = next_node;
    }
}

```

```

void SLL::print(void)
{
    cout << endl << " Printing list...." << endl;
    if (list_size == 0)
    {
        cout << " The list is empty!!" << endl;
    }
    else
    {
        Node* current_node = starting_node;
        for (long i = 1; i <= list_size; i++)
        {
            cout << " " << current_node->marks << ", ";
            current_node = current_node->next;
        }
        cout << endl;
    }
    cout << endl;
}

void SLL::push_front(float marks_to_add)
{
    cout << " Adding to front: " << marks_to_add << endl;
    if (list_size == 0)
    {
        starting_node = new Node;
    }
    else
    {
        Node* new_node = new Node;
        new_node->next = starting_node;
        starting_node = new_node;
    }
    starting_node->marks = marks_to_add;
    list_size++;
}

int main()
{
    SLL L;
    L.print();
    L.push_front(50);
    L.push_front(40);
    L.push_front(30);
    L.push_front(20);
    L.push_front(10);
    L.push_back(60);
    L.push_back(70);
    L.push_back(80);
    L.push_back(90);
    L.push_back(100);
    L.print();
    L.swap_all();
    L.print();
    cout << " Marks at index 3: " << L.at(3) << endl << endl;
    cout << " Deleting marks at index 3: " << L.at(3) << endl;
    L.delete_kth(3);
    L.print();
    return 0;
}

```